

BN NHWC 设计与实现

 已关联2个iCafe卡片

前向

Batchnorm 背景

[BN agroup 文档](#)

功能

$$\mu_i = \frac{1}{nhw} \sum_{j=1}^{nhw} x_{i,j}$$

$$\sigma_i^2 = \frac{\sum_{j=1}^{nhw} (x_{i,j} - \mu_i)^2}{nhw}$$

$$y_{i,j} = \frac{x_{i,j} - \mu_i}{\sqrt{\sigma_i^2 + eps}} * scale_i + bias_i$$

$$y = act(y + branch)$$

$$global_mean = momentum * global_mean + (1 - momentum) * batch_mean$$

$$global_var = momentum * global_var + (1 - momentum) * batch_var$$

其中 $i \sim (0, c)$, $j \sim (0, n * h * w)$ 。上述公式对每一个channel的数据做normalization。实际访存的pattern取决于数据的格式是nchw或者nhwc。

从 Batchnorm 算子的功能上分析可知，bn 主要做的事情有：

1. 对输入数据 x 的 c 纬度数据进行累加和平方累加。
2. 利用 c 维的累加数据对 x 进行计算。

BN NHWC 前向设计

对 nhwc 格式的数据来说，可将其视为 $nhw * c$ ，每行长度为 c，共 nhw 行。

天然的可以使用 simd 进行计算加速。

又由于需要对所有 c 维数据进行累加，而 cluster 之间没有好的同步办法，因此需要对前向拆成两个 phase。

phase1

phase1 的目标是求 `sum(x)` 和 `square_sum(x)`

因此主要考虑 DMA 效率。经过设计，共分为三种策略

策略一：兜底策略

</>

C++ | 收起 ^

```
1  for (cur_c_start = 0; cur_c_start < c; cur_c_start += capability) {
2      cur_c_len = min(TOTAL_BUFFER_SIZE, c - cur_c_start);
3      // int group_size = c / TOTAL_BUFFER_SIZE;
4      //[ sum ][align_32][ square_sum ]
5      float *buffer_square = buffer_out + roundup32(cur_c_len);
6
7      // set out buffer to zero
8
9      for (int iter = tid; iter < nhw; iter += nts) {
10         // for (int iter = pixel_start; iter < pixel_end; ++iter)
11         GM2LM(x_gm + iter * c + cur_c_start, buffer, cur_c_len * sizeof(T));
12         sum_square_sum_line(buffer, buffer_out, buffer_square, cur_c_len);
13     }
14
15     mfence();
16     sync_all();
17
18     // reduce all cores data to core 0
19
20     if (cid == 0) {
21         // flush data to GM
22     }
23 }
```

兜底策略首先对 `c` 进行切分，避免 `c` 过大超出 `buffer_size` 的情况。

每次读取一段 `c'` 进行计算，

将结果累加到 `core 0` 上，通过 `core 0` 刷到 GM 上。

策略二：当 `c` 较小时同时读取多行

进一步需要思考的点在于，一次性读取多行，在最后 `cluster` 内 `reduce_sum` 的时候，需要将多行累加成一行。

两种情况：

1) `c` 长度对 32 对齐，则可以通过 `simd` 在 `buffer_size` 内对多行 `c` 进行累加。

优点：

i. 计算很快，最后行内累加也可以用 simd。

缺点：

i. 对 c 要求较为苛刻，优化限制较大。

2) c 长度比较奇怪，无法保证 buffersize 内的连续 c 都可以用 simd 计算。

优点：

i. 对所有 $c < buffersize / 2$ 的规模都可以这么做。

缺点：

i. reduce 之后的行内加需要 sfu 计算，速度会慢一些。

综合来说两种情况都可以同时保留，实现时出于时间关系，先做 1)，另一个以后有时间再加。

策略三：当 c 较大时对 c 进行分段计算

这是需要考虑的点最多的策略。

当 c 过大，无法一次性放入 buffer 的时候，显然会考虑到将 c 分割计算，类似“策略一”，将每个 c 划分为 c’。

但出于 DMA 性能考虑，XPU2 GDDR 的读取要求所有 core 以 512 byte 在 GM 上连续读才能达到峰值性能。

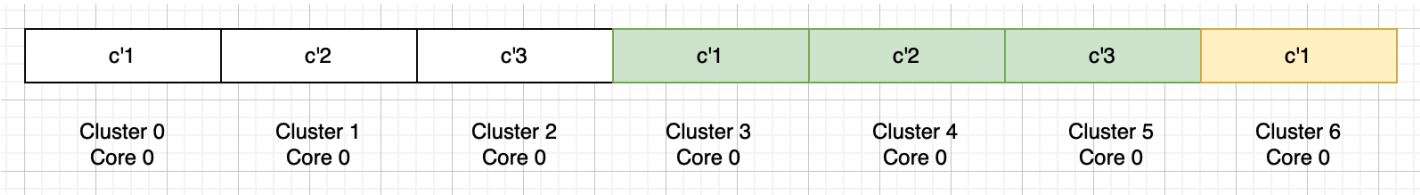
即：

```
int cur_c_idx;
int tid = cid * nclusters + clusterid;
int nts = ncores * nclusters;
if ((c <= TOTAL_BUFFER_SIZE / 2) && (c %
```

DMA 理论性能划分

利用如图方式对所有 core 进行划分，然后按照 tid 依次连续的对 GM 进行读取。

其问题在于最优化 DMA 加载 pattern 为：



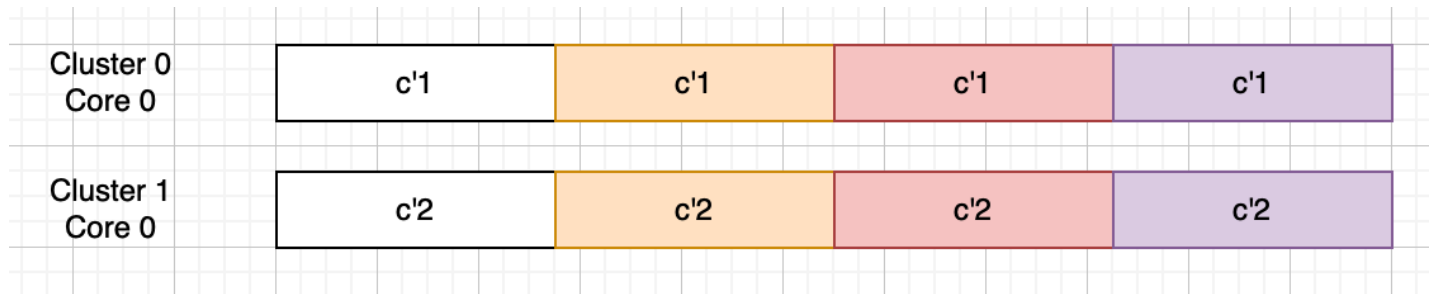
每段 c’ 的任务分配情况

令 c 划分为 3 段，分别为 c'1, c'2, c'3。如上图，第一轮每一个 core 处理一个 c’。

当 $nts \% nc' \neq 0$ 时，每一个 thread 每次处理的 c' 都是不同段，最终每个 thread 会处理整个 c 中的所有数据。

由于 c 较大，每个 thread 都处理整个 c 很容易导致 LM 不足，而这种算法无法再对 c 做拆分（除非牺牲性能），

因此，只有当 c 符合 $(nts \% nc' == 0)$ 时，可以通过拆 c 进行优化。



这种情况下，每个 thread 固定处理一类 c'

未来若存在模型就是 c 很大且不符合要求，可以尝试通过在每个 loop 读写 sm 应用这种优化，drawback 取决于读写 SM 的效率，前提是 SM 保证放得下。

phase 2

phase2 主要目的是对 phase1 的结果进行累加，然后利用 c 维数据对 x 进行处理。

需要注意的性能点有两个：

1. 对 phase1 结果进行累加处理
2. 对 x 的访问 pattern。

对1

phase1 的结果长度与 c 维长度和 cluster 数量有关，因此数据量相对极小。

为了稍微均匀分配计算任务，当前版本采用了在 core 之间对 c 划分

```
int partition_c_start, partition_c_end;
partition(cid, ncores, loop_c, 32, &partition_c_start, &partition_c_end);
```

计算完成后数据利用 simd 写到 SM

```
GM2LM_ASYNC(bias_gm + gm_c_iter, bias, len * sizeof(float));
primitive_mul(scale, sample_var, scale, len); // scale
primitive_mul(sample_mean, scale, sample_mean, len);
mfence_lm();
primitive_sub(bias, sample_mean, bias, len);
flush_to_sm(scale, scale_sm + c_iter, len); // partial c
flush_to_sm(bias, bias_sm + c_iter, len);
```

拷贝截图
快捷键

性能状况

bn_grad_nhwc 理论性能分析

对 BN 的性能优化从理论分析出发，

理论分析的依据为 BN 整体为 DMA 瓶颈，DMA 过程在整个算子中占比最大。

```
74 GM 带宽:
75 • no ecc 476 GB/s
76 • ecc 423 GB/s
77 • 颗粒质量不足 370 GB/s
78 • cluster 受限于 DMA 143 GB/s 读写全双工
79 • 8 cluster 总计 1144 GB/s
80
81 | 但是 8 cluster 一起算受限于 GM 性能, GM 只有 370.
82
83 phase1 读一次 写 c
84 phase2 读一次 读一次 c 写一次
85 理论规模读一次的性能是 (256 * 256 * 56 * 56) * 2 / 1024 / 1024 / 1024 / 370 * 1000ms = 1.0346ms
86 fp32 1.0346 * 2 = 2.0692ms
```

根据如图方式，可以计算到对 256，256，56，56 这一规模的 bn case，读一次 x 维度的 fp16 数据大约需要 1 ms，根据前向反向的计算 pattern（是否有 branch 等）就可以估计出算子的极限性能。

optv1 9.6															
loop	n	c	h	w	branch	relu	bitmask	phase1理论	phase1 实际	phase2 理论	phase2 实际	理论总时长	实际总时长		
35	6	256	1024	14	14	TRUE	TRUE	FALSE	0.25865709	0.284043	0.90529983	0.957127	7.44702		
36	1	256	1024	14	14	FALSE	FALSE	FALSE	0.25865709	0.284199	0.60353322	0.639184	0.923383		
37	7	256	128	28	28	FALSE	TRUE	FALSE	0.12932855	0.135233	0.30176661	0.325966	3.228393		
38	1	256	128	56	56	FALSE	TRUE	FALSE	0.51731419	0.526602	1.20706644	1.27654	1.803142		
39	3	256	2048	7	7	TRUE	TRUE	FALSE	0.12932855	0.155364	0.45264992	0.484353	1.919151		
40	1	256	2048	7	7	FALSE	FALSE	FALSE	0.12932855	0.155352	0.30176661	0.32324	0.478592		
41	11	256	256	14	14	FALSE	TRUE	FALSE	0.06466427	0.08907	0.15088331	0.167171	2.818651		
42	1	256	256	28	28	FALSE	TRUE	FALSE	0.25865709	0.283542	0.60353322	0.649209	0.932751		
43	3	256	256	56	56	TRUE	TRUE	FALSE	1.03462838	1.066307	3.62119932	3.825196	14.674509		
44	1	256	256	56	56	FALSE	FALSE	FALSE	1.03462838	1.063736	2.41413288	2.55198	3.615716		
45	1	256	512	14	14	FALSE	TRUE	FALSE	0.12932855	0.154296	0.30176661	0.32826	0.482556		
46	4	256	512	28	28	TRUE	TRUE	FALSE	0.51731419	0.542336	1.81059966	1.922349	9.85874		
47	1	256	512	28	28	FALSE	FALSE	FALSE	0.51731419	0.5442	1.20706644	1.277063	1.821263		
48	5	256	512	7	7	FALSE	TRUE	FALSE	0.03233214	0.057272	0.07544165	0.087905	0.725885		
49	1	256	64	112	112	FALSE	TRUE	FALSE	1.03462838	1.054952	2.41413288	2.549687	3.604639		
50	6	256	64	56	56	FALSE	TRUE	FALSE	0.25865709	0.264124	0.60353322	0.641043	5.431002		
51													59.765393		

实际测试表明，nhwc 相对 nchw 性能更优，占用 L3 更少。

反向

反向的数据访问情况与前向很像，因此优化策略也是上面这三种。

L3 策略分析

对于 nchw 格式，由于对 x 维数据的读取特别麻烦，因此利用 L3 缓存 x 维数据总是能获得最佳性能。且由于 c 维连续的关系，只需要一部分数据，就能算出某个或者某几个 c 维的累加和，因此可以循环放到 I3 上计算 bn 的整个流程。

为此，在反向计算累加和时会对 dy 进行 relu grad 操作，并将累加结果保存到 dbranch 或者虚构的 I3 buffer 中去。在 反向第二阶段从 I3 直接读 relu_grad_dy 可以少一次计算。

但是在 nhwc 格式中，策略会变得更加复杂。因为更多的数据都是放在 GM 的。

1. 无 bitmask

- a. `load dy & y + load dy & y = 4 * l(x) + 2 * c(x);` // `l(x)` for `load(x维)` `c(x)` for `calculate(x维)`
- b. `load dy & y, store relu_grad_dy + load relu_grad_dy = 3 * l(x) + 1 * s(x) + 1 * c(x); = 4.2 * l(x) + 1 * c(x)`
 - i. 当 `x` 能放到 L3 时可以加速。

2. 有 bitmask

- a. `load dy & bitmask + load dy & bitmask = 2 * l(x) + 2 * l(mask) + 2 * c(x)`
- b. `load dy & bitmask, store relu_grad_dy + load relu_grad_dy = 2 * l(x) + 1 * s(x) + 1 * l(mask) + 1 * c(x)`
 - i. 明显慢于 a, 相当于一次 `store` 换 `1 * l(mask) + 1 * c(x)`

上述4种情况, 有一些是需要测试的, 有一些是明显偏慢的, 都需要后续仔细考虑。

推算可得:

一次 `bitmask` 为 $41.45 - 36.95 = 4.5\text{ ms}$

`bitmask` 比 `y` 快 $41.32 - 36.95 = 4.37\text{ ms}$

一次 `y` = 8.87 ms

`c(x)` 怎么都到不了 4ms , 所以圈里这个是明显慢于 a 的结论是正确的。

经过测试, 有 `bitmask` 的情况下, 无论是否有 L3 空间, 都可以用 2.a 策略。

一些总结

- 1) 注意 DMA 最优的众核工作策略, 在此基础上可以通过理论分析代码的好坏。对 DMA 瓶颈算子来说, 数据 `load` 的 `pattern` 大于一切。
- 2) 感知了硬件的一些特性。
 - a. `for loop` 每次 512 byte 读是最快的。
 - b. 若在一次 `loop` 中要读多个数据, 则性能会稍有影响。 $1 + 1 \sim = 2.2$
 - c. GDDR 写会比读慢一些, 约为 1.2 倍。
 - d. row conflict 在单测时有发生, 性能影响大约在 5%, 调整数据的 `malloc` 位置可以看到明显且稳定的性能变化。